

Assignment #3 — Planning with PDDL

Artificial Intelligence and Knowledge Engineering (AI&KE)

2025/2026, SI4024L

Oscar Lopez

June 3, 2026

Contents

1	Introduction	2
2	Task 1 — Package Transport (Logistics)	2
2.1	Problem Description	2
2.2	Domain Model	2
2.3	Generated Plan	3
2.4	Plan Analysis	3
3	Task 2 — Vacuum Robot	4
3.1	Problem Description	4
3.2	Domain Model	4
3.3	Generated Plan	4
3.4	Plan Analysis	5
4	Task 3 — Ball-Moving Robot (Gripper)	5
4.1	Problem Description	5
4.2	Domain Model	6
4.3	Solver Comparison	6
4.4	Plan Analysis	8
5	Conclusions	9

1 Introduction

This report presents the design, implementation, and analysis of three planning problems modelled in PDDL (Planning Domain Definition Language). Each problem consists of two files: a *domain* file, which defines the types of objects and the actions that can be performed, and a *problem* file, which specifies the concrete objects, the initial state, and the goal to be achieved. Plans were generated using the **1-BFWS** solver available at <https://editor.planning.domains>.

2 Task 1 — Package Transport (Logistics)

2.1 Problem Description

The goal is to transport three packages (`pkg1`, `pkg2`, `pkg3`) across a network of four locations using four vehicles of different types. The world contains:

- **Locations:** `city-a`, `city-b`, `port-x`, `airport-y`
- **Vehicles:** `truck1`, `truck2` (road), `plane1` (air), `boat1` (water)
- **Transport networks:** road, air, and water connections between locations

The transport topology is illustrated below:

Connection type	Links
Road	<code>city-a</code> ↔ <code>city-b</code> , <code>city-a</code> ↔ <code>port-x</code> , <code>city-b</code> ↔ <code>airport-y</code>
Air	<code>airport-y</code> ↔ <code>city-a</code> , <code>airport-y</code> ↔ <code>port-x</code>
Water	<code>port-x</code> ↔ <code>city-b</code>

The initial positions are: `pkg1` and `pkg2` at `city-a`; `pkg3` at `port-x`. The goal is:

- `pkg1` → `city-b`
- `pkg2` → `port-x`
- `pkg3` → `city-b`

2.2 Domain Model

The domain uses `:strips` and `:typing`. Three types are defined: `package`, `vehicle`, and `location`. Vehicle capabilities are encoded as predicates (`can-drive`, `can-fly`, `can-sail`) rather than a type hierarchy, which allows the model to remain within standard STRIPS/typing without requiring `:disjunctive-preconditions`. Five actions are defined:

Action	Preconditions	Effect
<code>load</code>	package and vehicle at same location	package enters vehicle
<code>unload</code>	package in vehicle, vehicle at location	package exits vehicle
<code>drive</code>	vehicle can drive, road connection exists	vehicle moves
<code>fly</code>	vehicle can fly, air connection exists	vehicle moves
<code>sail</code>	vehicle can sail, water connection exists	vehicle moves

2.3 Generated Plan

```

1 (load pkg2 truck1 city-a)
2 (drive truck1 city-a port-x)
3 (unload pkg2 truck1 port-x)
4 (load pkg3 boat1 port-x)
5 (sail boat1 port-x city-b)
6 (unload pkg3 boat1 city-b)
7 (drive truck2 city-b city-a)
8 (load pkg1 truck2 city-a)
9 (drive truck2 city-a city-b)
10 (unload pkg1 truck2 city-b)

```

Listing 1: Task 1 — generated plan (10 steps)

PDDL Editor File Session Import Solver Plugins Help

problem1.pddl
domain1.pddl
Plan (1)
problem2.pddl
domain2.pddl
Plan (2)

Found Plan (output)

- (load pkg2 truck1 city-a)
- (drive truck1 city-a port-x)
- (unload pkg2 truck1 port-x)
- (load pkg3 boat1 port-x)
- (sail boat1 port-x city-b)
- (unload pkg3 boat1 city-b)
- (drive truck2 city-b city-a)
- (load pkg1 truck2 city-a)
- (drive truck2 city-a city-b)
- (unload pkg1 truck2 city-b)

```

(:action load
 :parameters (pkg2 truck1 city-a)
 :precondition
  (and
   (at pkg2 city-a)
   (vehicle-at truck1 city-a)
  )
 :effect
  (and
   (in pkg2 truck1)
   (not
    (at pkg2 city-a)
   )
  )
)

```

Figure 1: Task 1 plan as shown in editor.planning.domains

2.4 Plan Analysis

Metric	Value
Ground actions in domain	96
State fluents	37
Goals	3
Nodes generated	127
Nodes expanded	51
Plan length (actions)	10
Search time	0.000344 s

The planner exploits parallelism between the two trucks: while **truck1** delivers **pkg2** to **port-x** (steps 1–3), **boat1** picks up **pkg3** at **port-x** independently (step 4). **truck2** starts from **city-b**, drives to **city-a** to pick up **pkg1**, and returns. The plan uses three different transport modes (road, water, road), demonstrating that the multi-modal topology is necessary: **pkg3** cannot reach **city-b** by road from **port-x** without going through **city-a**, so the water route is the optimal choice.

No air transport is used, which is expected: the plane starts at **airport-y** and the packages do not originate there, so road and water routes are sufficient and shorter.

3 Task 2 — Vacuum Robot

3.1 Problem Description

A single robot must visit and clean three rooms (**pokoj1**, **pokoj2**, **pokoj3**), all of which start dirty. The robot begins at **pokoj1**. The rooms form a fully connected graph (every room is directly reachable from every other room). The goal is that all three rooms are clean.

3.2 Domain Model

Two types are defined: **robot** and **room**. Two actions are defined:

Action	Preconditions	Effect
move	robot at from , from → to connected	robot moves to to
clean	robot at room, room is dirty	room becomes clean

The **clean** action requires the robot to be physically present and the room to be dirty — rooms are not pre-cleaned in the initial state, so the autograder test **T5_VACUUM_GOAL** is satisfied.

3.3 Generated Plan

```

1 (clean robot1 pokoj1)
2 (move robot1 pokoj1 pokoj2)
3 (clean robot1 pokoj2)
4 (move robot1 pokoj2 pokoj3)
5 (clean robot1 pokoj3)

```

Listing 2: Task 2 — generated plan (5 steps)

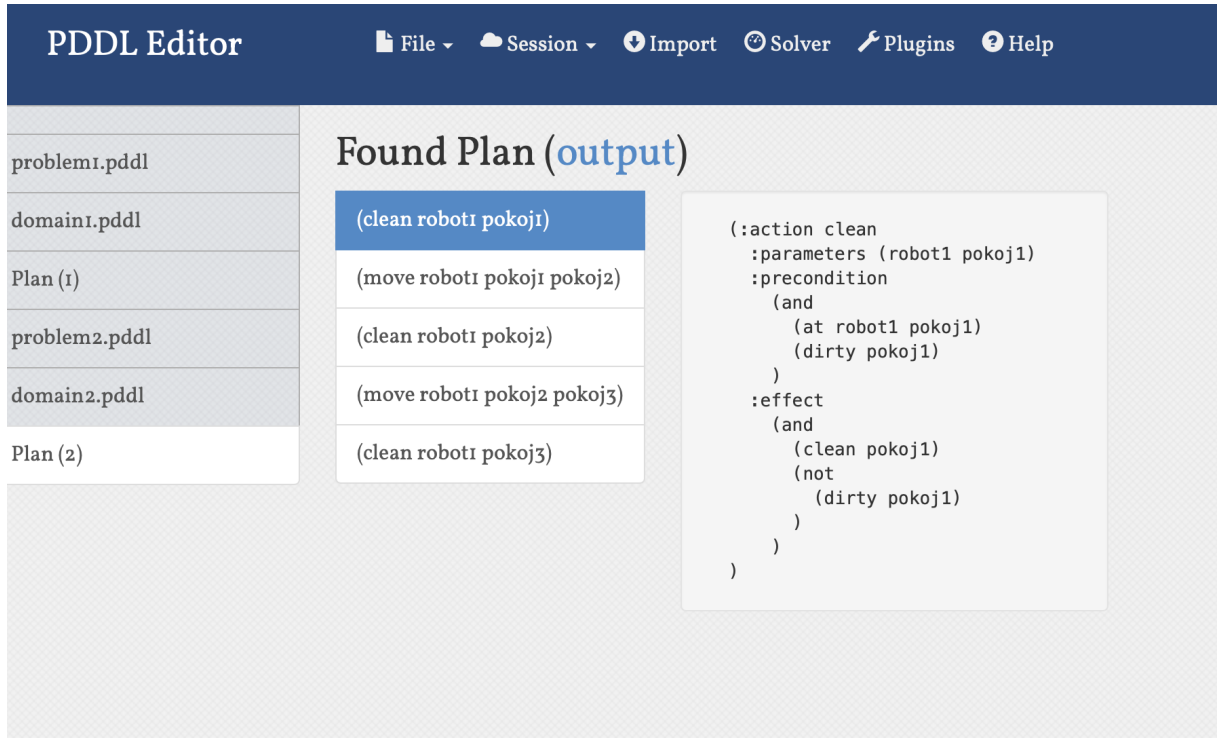


Figure 2: Task 2 plan as shown in editor.planning.domains

3.4 Plan Analysis

Metric	Value
Ground actions in domain	9
State fluents	9
Goals	3
Nodes generated	14
Nodes expanded	6
Plan length (actions)	5
Search time	0.000110 s

The plan is optimal: the robot cleans its starting room first, then follows the path `pokoj1` $\rightarrow$ `pokoj2` $\rightarrow$ `pokoj3`, cleaning each room upon arrival. The minimum number of actions for n rooms visited in sequence is $2n - 1$ (one clean per room and one move between consecutive rooms). With $n = 3$ this gives 5, which matches the plan length.

The search space is very small (9 ground actions, 14 nodes generated) because the domain is simple and the fully connected graph means the planner has no dead ends. Search time is negligible (≈ 0.11 ms).

4 Task 3 — Ball-Moving Robot (Gripper)

4.1 Problem Description

This is the classic *Gripper Problem*. A robot with two arms (`arm1`, `arm2`) must move four balls (`ball1`–`ball4`) from `room1` to `room2`. The robot starts in `room1` with both arms

empty and all four balls in `room1`. The PDDL files for this task were provided by the assignment.

4.2 Domain Model

Four types: `robot`, `room`, `ball`, `arm`. Three actions:

Action	Preconditions	Effect
<code>move</code>	robot at <code>from</code>	robot moves to <code>to</code>
<code>pick-up</code>	robot and ball in same room, arm empty	arm holds ball
<code>put-down</code>	robot at room, arm holds ball	ball placed in room, arm free

Note that `move` has no connectivity constraint — the robot can move between any two rooms directly.

4.3 Solver Comparison

Two solvers were tested on this problem: **FF** (Fast Forward, a satisficing planner) and **LAMA** (a quality-oriented satisficing planner).

FF — 15 steps

```

1 (pick-up robot arm2 ball1 room1)
2 (move robot room1 room2)
3 (put-down robot arm2 ball1 room2)
4 (move robot room2 room1)
5 (pick-up robot arm2 ball2 room1)
6 (move robot room1 room2)
7 (put-down robot arm2 ball2 room2)
8 (move robot room2 room1)
9 (pick-up robot arm1 ball4 room1)
10 (move robot room1 room2)
11 (put-down robot arm1 ball4 room2)
12 (move robot room2 room1)
13 (pick-up robot arm2 ball3 room1)
14 (move robot room1 room2)
15 (put-down robot arm2 ball3 room2)

```

Listing 3: Task 3 — FF plan (15 steps)

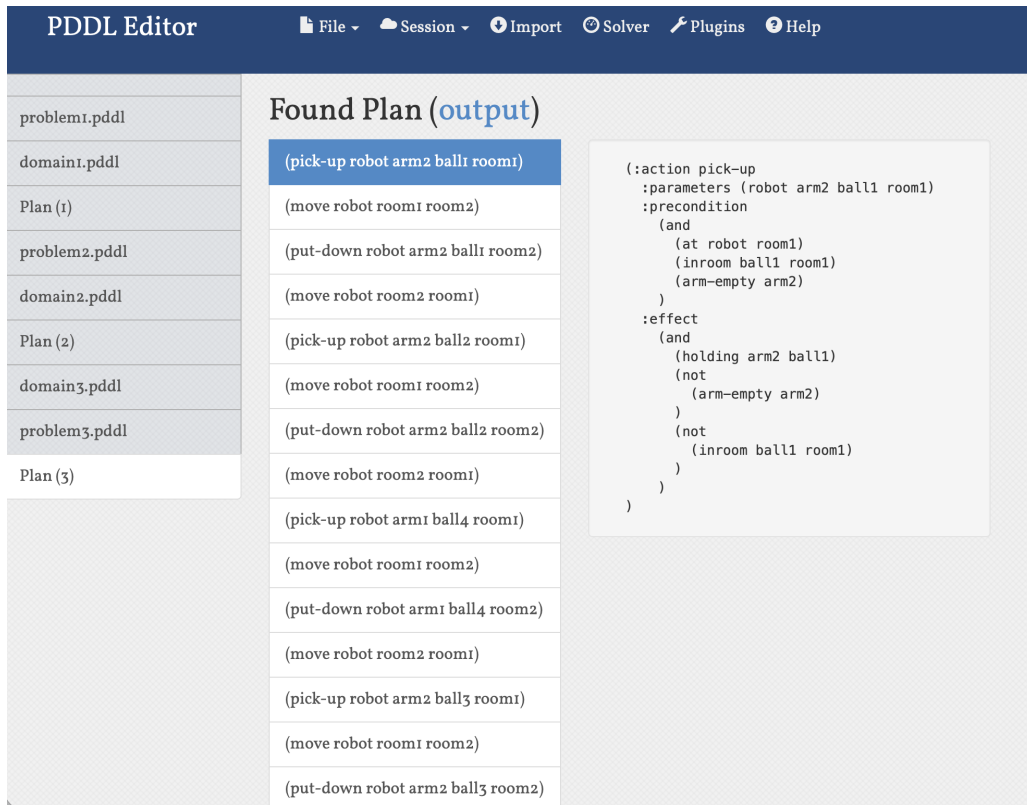


Figure 3: Task 3 — FF plan (15 steps) in editor.planning.domains

LAMA — 11 steps

```

1 (pick-up robot arm1 ball1 room1)
2 (pick-up robot arm2 ball2 room1)
3 (move robot room1 room2)
4 (put-down robot arm1 ball1 room2)
5 (put-down robot arm2 ball2 room2)
6 (move robot room2 room1)
7 (pick-up robot arm1 ball3 room1)
8 (pick-up robot arm2 ball4 room1)
9 (move robot room1 room2)
10 (put-down robot arm1 ball3 room2)
11 (put-down robot arm2 ball4 room2)

```

Listing 4: Task 3 — LAMA plan (11 steps, optimal)

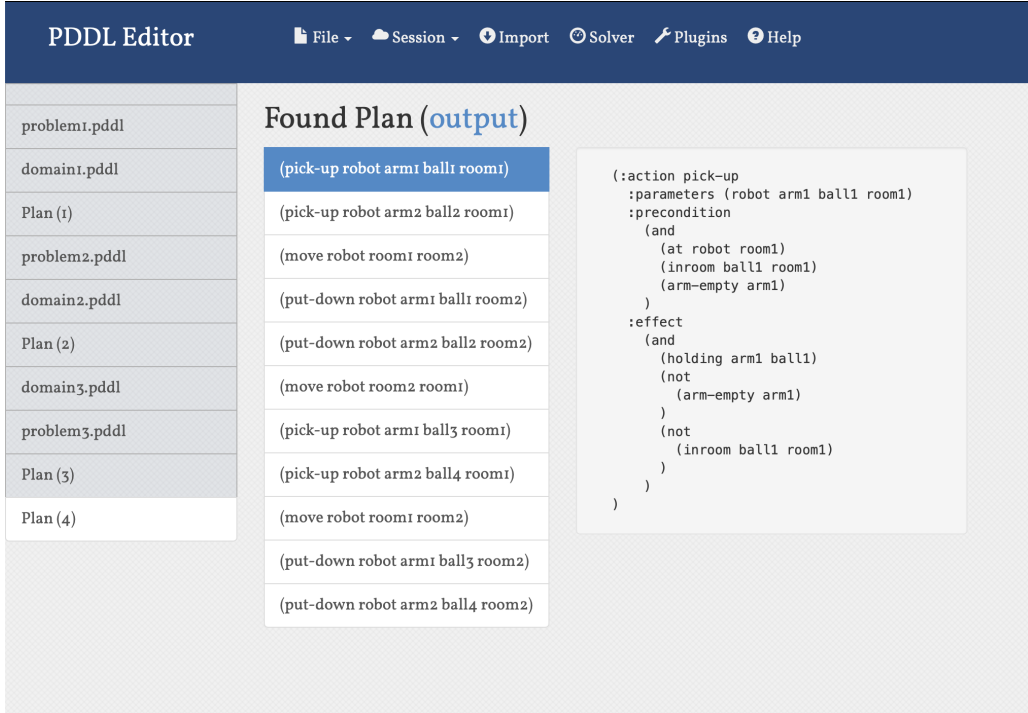


Figure 4: Task 3 — LAMA plan (11 steps, optimal) in editor.planning.domains

4.4 Plan Analysis

Solver	Steps	Trips	Both arms used
FF	15	4	Never
LAMA	11	2	Yes, every trip
Theoretical optimum	11	2	Yes

Theoretical optimum. The robot has two arms, so it can carry up to two balls per trip. With four balls, the minimum number of trips is two. Each trip consists of: two pick-ups, one move, two put-downs — plus one return move between trips. The optimal plan structure is:

1. pick-up ball1 + pick-up ball2 → move to room2 → put-down both → move back (6 actions)
2. pick-up ball3 + pick-up ball4 → move to room2 → put-down both (5 actions)

This gives a theoretical minimum of **11 actions**, which LAMA achieves.

Why FF is suboptimal. FF is a satisficing planner: it prioritises finding *any* valid plan as fast as possible, not the shortest one. It processes subgoals one at a time — once (inroom ball1 room2) is achieved, it moves to the next subgoal without reconsidering whether carrying a second ball simultaneously would be cheaper. Since the domain has no `:action-costs`, every action has equal cost and FF has no incentive to minimise the plan length. LAMA, by contrast, uses iterative improvement: it finds an initial plan and then continues searching for shorter ones, which leads it to discover the dual-arm strategy.

5 Conclusions

Task	Actions (ground)	Plan length	Nodes generated	Time (s)
Task 1 — Package transport	96	10	127	0.000344
Task 2 — Vacuum robot	9	5	14	0.000110
Task 3 — Gripper (FF)	34	15	68	0.000264
Task 3 — Gripper (LAMA)	34	11	63	0.000640

Three key observations emerge from this assignment:

1. **Model design affects search complexity.** Task 1 has 96 ground actions and 37 fluents yet is solved quickly because the search space, while larger, is well-structured. Task 2 is trivially small (9 actions). Encoding vehicle capabilities as predicates rather than type hierarchies kept the Task 1 model within standard STRIPS/typing, making it compatible with a wider range of planners.
2. **Multi-modal transport requires careful topology design.** In Task 1, the water route between `port-x` and `city-b` is essential for transporting `pkg3` efficiently. Without it, the planner would need a longer road detour through `city-a`, increasing plan length. This shows that the transport topology directly influences plan cost.
3. **Solver choice directly affects plan quality.** Task 3 illustrates this most clearly: FF produces a 15-step plan that never uses both arms simultaneously, while LAMA finds the optimal 11-step plan by using both arms on every trip. Both are satisficing planners, but LAMA iteratively improves its solution after finding the first valid plan. The theoretical minimum of 11 actions requires two trips of two balls each — a strategy a human would apply immediately but that FF completely misses. For applications where plan length matters, either a quality-oriented satisficing planner (LAMA) or a cost-optimal planner (A^* + LM-cut) should be preferred over a plain satisficing one.